In [ ]:
```
import marimo as mo
from z3 import (
    BitVec,
    And,
    Or,
    Not,
    ULT,
    UGE,
    LShR,
    Solver,
    sat,
    unsat,
    ZeroExt,
)
import unittest
n=8 #parametro
```

Exercício 3: Multiplicação de inteiros como SFOTS

Introdução

Este exercício parte de um programa imperativo que calcula a multiplicação de dois inteiros não negativos, a e b, usando apenas deslocamentos, somas e subtrações. O programa guarda três variáveis, x, y, z, que começam com os valores $a, b, 0$. Enquanto y for diferente de zero, repete-se um ciclo:

- se y é par, duplica-se x e divide-se y por 2;
- se y é ímpar, subtrai-se um a y e soma-se x a z.

Quando y chega a zero, o programa termina e z deve ser igual a $a \cdot b$.

A precisão é limitada a n bits, parâmetro do programa. Por isso tanto o deslocamento de x como a soma $z + x$ podem dar **overflow**. Isto não se vê no código, mas o diagrama mostra-o.

O modelo: Control Flow Automaton

O `prob3.pdf` descreve o programa como um CFA (*Control Flow Automaton*), um grafo orientado onde:

- os **nodos** são ações (`init` , `skip` , `left` , `right` , `stop` e `error`) que atuam sobre os valores das variáveis;
- os **ramos** transferem o resultado de um nodo para o seguinte, condicionados por uma condição (a ausência da mesma equivale a `True`);

- se um ramo leva a **erro**, tem **precedência** sobre os outros; se não houver precedência, a escolha é **não determinística**;
- o **estado** do sistema é o identificador do nodo mais o valor das variáveis no *input* desse nodo.

A partir de `skip` há três saídas possíveis: para `left` se $y \neq 0$ e y é par, para `right` se $y \neq 0$ e y é ímpar, e para `stop` se $y = 0$. Tanto `left` como `right` podem ir para `error` em caso de overflow. O diagrama desenha dois nodos `error` (um por ramo), mas no modelo usa-se um só.

O que se pede

Traduzir este autómato num SFOTS em Z3, usando `BitVec` de tamanho n, e depois verificar propriedades sobre ele:

- **a)** construir o SFOTS: identificar as variáveis do modelo, o estado inicial, a relação de transição e o estado de erro;
- **b)** verificar por k-indução que $x \cdot y + z = a \cdot b$ é um invariante;
- **c)** verificar a segurança do programa: com $a < 2^{n/2}$ e $b < 2^{n/2}$ no estado inicial, o estado de erro nunca é acessível;
- **d)** codificar a propriedade de animação que exprime que o programa termina quaisquer que sejam a e b, e prová-la no SFOTS construído.

Implementação completa

In [ ]:
```
def declare(i): #declara estado s, declaro(0) da atual e o 1 o seguinte
    s={}
    s['pc'] = BitVec('pc' + str(i), n) #program counter, em que nodo se esta
    s['a'] = BitVec('a' + str(i), n) #a e b n mudam durante a execucao mas defino
agora
    s['b'] = BitVec('b' + str(i), n)
    s['x'] = BitVec('x' + str(i), n)
    s['y'] = BitVec('y' + str(i), n)
    s['z'] = BitVec('z' + str(i), n)
    return s

def init(s): #estado inicial, a e b livres ainda
    return And(s['pc'] == 0, s['x'] == s['a'], s['y'] == s['b'], s['z'] ==0)

def keep(s, t, nomes):
    return And([t[v] == s[v] for v in nomes])

def skip_left(s,t):
    return And(s['pc'] == 1, s['y'] !=0, (s['y'] & 1)==0,
    t['pc'] ==2,
    keep(s,t, ['a','b','x','y','z']))

def skip_stop(s,t):
    return And(s['pc'] == 1, s['y']==0,
```

```
            t['pc'] == 5,
            keep(s,t,['a','b','x','y','z']))

def left_skip(s,t):
    return And(s['pc'] == 2,
            ULT(s['x'], 2**(n-1)), #sem overflow no shift
            t['pc'] ==1 ,
            t['x'] == s['x'] << 1,
            t['y'] == LShR(s['y'],1), #div por 2 sem sinal
            keep(s,t,['a', 'b' , 'z']))

def left_error(s,t):
    return And(s['pc'] ==2,
    UGE(s['x'], 2**(n-1)), #ha overflow
    t['pc'] == 4,
    keep(s,t,['a','b','x','y','z']))

def right_skip(s, t):
    return And(s['pc'] == 3,
               UGE(s['z'] + s['x'], s['z']),   #sem overflow na soma
               t['pc'] == 1,
               t['y'] == s['y'] - 1,
               t['z'] == s['z'] + s['x'],
               keep(s, t, ['a','b','x']))

def right_error(s, t):
    return And(s['pc'] == 3,
               ULT(s['z'] + s['x'], s['z']),   # verflow na soma
               t['pc'] == 4,
               keep(s, t, ['a','b','x','y','z']))

def erro(s):
    return s['pc'] == 4

def init_skip(s,t):
    return And(s['pc'] == 0, t['pc'] == 1,
            keep(s, t, ['a','b','x','y','z']))

def skip_right(s,t):
    return And(s['pc'] == 1, s['y'] != 0, (s['y'] & 1) == 1,
            t['pc'] == 3,
            keep(s, t, ['a','b','x','y','z']))

def ramos(s,t):
    return Or(init_skip(s,t), skip_left(s,t), skip_stop(s,t), left_skip(s,t),
left_error(s,t), skip_right(s,t), right_skip(s,t), right_error(s,t))

def terminal(s):
    return Or(s['pc'] == 4, s['pc'] == 5)

def laco(s,t):
    return And(terminal(s), keep(s,t,['pc','a','b','x','y','z']))

def trans(s,t):
    return Or(ramos(s,t),laco(s,t))
```

```
def induction_always(declare, init,trans,inv):
    s,t=declare(0), declare(1)
    sol=Solver()
    sol.add(init(s),Not(inv(s)))
    if sol.check() == sat:
        return "Base falha"
    sol= Solver()
    sol.add(inv(s), trans(s,t),Not(inv(t)))
    if sol.check() == sat:
        return "Passo Indutivo falha"
    return "invariante"

def Kinduction_always(declare, init,trans, inv,k):
    st=[declare(i) for i in range(k+1)]
    sol=Solver()
    sol.add(init(st[0]))
    for i in range (k-1):
        sol.add(trans(st[i],st[i+1]))
    sol.add(Or([Not(inv(st[i]))for i in range(k)]))
    if sol.check()== sat:
        return "Base falha"
    sol=Solver()
    for i in range(k):
        sol.add(inv(st[i]), trans(st[i],st[i+1]))
    sol.add(Not(inv(st[k])))
    if sol.check() == sat:
        return "Passo Indutivo falha"
    return "invariante"

def P(s):
    return s['x']*s['y'] + s['z'] == s['a']*s['b']

def init_c(s):
    return And(init(s), ULT(s['a'], 2**(n//2)), ULT(s['b'], 2**(n//2)))

def W(v):
    return ZeroExt(n, v)

def Q(s):
    return And(Not(erro(s)),
               ULT(s['a'], 2**(n//2)), ULT(s['b'], 2**(n//2)),
               W(s['x'])*W(s['y']) + W(s['z']) == W(s['a'])*W(s['b']))

def caminho(K, inicio, final=None):
    st=[declare(i) for i in range(K+1)]
    sol=Solver()
    sol.add(inicio(st[0]))
    for i in range(K):
        sol.add(trans(st[i],st[i+1]))
    if final:
        sol.add(final(st[K]))
    return sol.check()

def V(s):
    y=W(s['y'])
    w=If(s['pc']==0, BitVecVal(3,2*n),
```

```
        If(s['pc']==1, BitVecVal(2,2*n), BitVecVal(1,2*n)))
    return If(terminal(s), BitVecVal(0,2*n), 4*y+w)

def decresce(s,t):
    return Or(ULT(V(t),V(s)),V(t)==0)

def decresce_forall(s):
    t=declare('p')
    return ForAll([t[v] for v in t], Implies(trans(s,t), decresce(s,t)))

def v_zero(s):
    return Implies(V(s)==0,terminal(s))
```

a) Construção do SFOTS

O sistema é modelado através de estados. Cada estado é constituído pelo valor do `pc`, que indica o nodo em que estamos no diagrama, e pelos valores das variáveis definidas em `declare`. Todas as variáveis são `BitVec` de n bits, ou seja, vetores de bits de tamanho fixo.

O `pc` identifica os diferentes nodos do sistema:

pc	Nodo
0	`init`
1	`skip`
2	`left`
3	`right`
4	`error`
5	`stop`

Além de `pc`, `x`, `y` e `z`, o estado inclui `a` e `b`. Estes não mudam durante a execução, mas é preciso guardá-los para se poder falar do valor $a \cdot b$ nas propriedades.

Estado inicial: $pc = 0$, $x = a$, $y = b$, $z = 0$. Começamos no nodo `init`, com os dois operandos da multiplicação guardados em x e y e o acumulador z a zero. Os valores de a e b ficam livres.

Relação de transição

A relação `trans` representa todas as transições possíveis entre um estado atual `s` e o estado seguinte `t`. Cada seta do diagrama é representada por uma fórmula lógica que define:

- o nodo de origem;

- a condição necessária para executar a transição;
- o nodo de destino;
- as alterações efetuadas às variáveis.

O `keep` é importante porque garante que as variáveis que não são modificadas numa transição mantêm o seu valor. Sem essa restrição, o Z3 poderia atribuir valores arbitrários no estado seguinte.

Cada transição individual combina as suas condições com `And`, e `trans` junta todas com um `Or`, porque representa as transições alternativas do sistema.

Estado de erro e deteção de overflow

O estado de erro é identificado por $pc = 4$.

As operações sobre `BitVec` no Z3 são feitas com um número fixo de bits. Os valores dão a volta, seguindo a aritmética módulo 2^n, sem que o Z3 indique que ocorreu overflow. Por isso, o overflow tem de ser detetado explicitamente.

No nodo `left`: ao duplicar x, podemos perder o bit mais significativo. Para detetar essa situação verificamos se $x \geq 2^{n-1}$, usando a comparação sem sinal `UGE`.

No nodo `right`: ao somar $z + x$, pode haver overflow se o resultado der a volta. Deteta-se verificando se a soma é inferior a z (comparação sem sinal, `ULT`).

Escolhemos a deteção sem sinal (`ULT`, `UGE` e `LShR`) porque o modelo do PDF trabalha com valores sem sinal e porque a condição da alínea c) pressupõe essa interpretação.

As condições que permitem continuar a execução e as que levam ao erro são **complementares**: se não há overflow, segue-se o ramo normal; se há, só o ramo de erro fica disponível. Assim, o erro tem precedência sobre a execução normal, como o PDF exige.

Estados `stop` e `error`

Nenhuma das 8 transições tem origem em $pc = 4$ ou $pc = 5$. Ou seja, `stop` e `error` **não têm transições de saída**: quando a execução lá chega, fica bloqueada. Todos os outros nodos têm sempre pelo menos uma transição possível. Este facto é usado na alínea d).

Como ler o código

O código não executa o autómato, descreve-o em lógica, e o Z3 procura valores que cumpram essa descrição. Cada ramo é uma fórmula com três partes:

- onde estamos: `s['pc'] == 1`;
- condição: `s['y'] != 0` e y par;
- como fica o seguinte: `t['pc'] == 2`, com as outras variáveis iguais.

`trans` é um `Or` de todos os ramos: para um par `(s, t)` ser válido, tem de cumprir um deles. O "saber para onde ir" funciona assim: em `skip` com $y = 2$, só o ramo `skip_left` tem a condição verdadeira, por isso o único `t` possível tem $pc = 2$. As condições de saída de `skip` (y par, y ímpar, $y = 0$) nunca se sobrepõem, e é isso que faz o autómato comportar-se como um programa determinista.

b) Invariante por k-indução

Pretende-se verificar se a propriedade

$$x \cdot y + z = a \cdot b$$

é um invariante do sistema, ou seja, se se mantém verdadeira ao longo da execução.

A ideia intuitiva é que o sistema vai transformando a multiplicação numa sequência de operações mais simples, sem alterar o valor total que se pretende calcular.

- No nodo `left`, x duplica e y é dividido por dois. Como y é par neste ramo, o produto $x \cdot y$ mantém-se: $2x \cdot \frac{y}{2} = x \cdot y$.
- No nodo `right`, y diminui uma unidade e z aumenta em x: $x(y-1) + (z+x) = xy - x + z + x = xy + z$.

A propriedade é então preservada pelas operações do algoritmo.

Para o verificar formalmente usa-se a k-indução, em duas etapas:

- **Base:** a propriedade é verdadeira nos primeiros k estados a partir do estado inicial.
- **Passo indutivo:** se a propriedade é verdadeira em k estados consecutivos, continua verdadeira no seguinte.

Em ambas se usa o Z3 para procurar um contraexemplo. Se o resultado for `unsat`, não existe nenhum estado ou sequência de estados que cumpra as condições e viole a propriedade, logo a verificação é bem-sucedida.

Com $k = 1$ falha. O passo indutivo devolve `sat`, com um contraexemplo em que $pc = 2$ (nodo `left`) e y é **ímpar** (o valor concreto, por exemplo 113, depende do Z3). Este estado não é alcançável, porque a transição de `skip` para `left` exige y par. Mas com $k = 1$ o Z3 não tem informação sobre o estado anterior para deduzir isso, e considera estados que satisfazem as condições locais mesmo sem serem alcançáveis a partir do estado inicial.

Com $k = 2$ passa. O passo indutivo passa a incluir o estado anterior ao `left`, que é `skip`, onde a condição de entrada em `left` exige y par. O contraexemplo deixa de ser possível.

c) Verificação da segurança

Pretende-se provar que, com $a < 2^{n/2}$ e $b < 2^{n/2}$ no estado inicial, o estado de erro nunca é acessível.

Porque é que não há overflow

Com a condição $a, b < 2^{n/2}$, o produto satisfaz $a \cdot b < 2^n$, ou seja, o resultado final cabe em n bits. Mas é preciso ver também as operações intermédias.

Nodo `left`. Duplica-se x e divide-se y por 2. A transição para `left` exige y par e não nulo, logo $y \geq 2$. Pelo invariante:

$$2x \leq x \cdot y \leq a \cdot b < 2^n$$

Assim, a duplicação de x não perde o bit mais significativo.

Nodo `right`. Soma-se x a z e decrementa-se y. Neste ramo $y \geq 1$, logo:

$$x + z \leq x \cdot y + z = a \cdot b < 2^n$$

Assim, a soma também não dá a volta.

Porque não basta provar $pc \neq 4$

Poderíamos tentar provar apenas $Q = (pc \neq 4)$, mas isso, por si só, não é k-indutivo. O Z3 pode considerar estados arbitrários que não correspondem a execuções reais, em que x e z têm valores elevados, permitindo overflow.

Por isso usa-se uma propriedade auxiliar Q, que combina três condições:

1. **Ausência de erro:** $pc \neq 4$.
2. **Limitação dos valores iniciais:** $a, b < 2^{n/2}$, garantindo que o produto cabe em n bits.
3. **Preservação do invariante:** $x \cdot y + z = a \cdot b$.

Outro cuidado: as multiplicações de `BitVec` têm tamanho fixo. Para verificar a igualdade $x \cdot y + z = a \cdot b$ sem perder informação por overflow **na própria verificação**, usa-se `ZeroExt` para estender os valores de n para $2n$ bits. Assim os produtos e somas têm espaço suficiente para comparar os valores matemáticos.

Falha com $k = 1$

Tal como na alínea b), com $k = 1$ o Z3 encontra um estado não alcançável. Por exemplo, um estado em `left` ($pc = 2$) com y ímpar e $x \geq 2^{n-1}$, que vai para `error`. Mesmo com o invariante incluído, a 1-indução não tem informação sobre o estado anterior para excluir estes estados artificiais. Com $k = 2$ já tem, e o passo indutivo passa.

d) Terminação (animação)

Pretende-se mostrar que, quaisquer que sejam a e b, o programa termina, isto é, a execução chega a `stop` ou a `error` e aí fica.

A ideia usada: como `stop` e `error` não têm transições de saída e todos os outros nodos têm sempre pelo menos uma, uma execução só pode parar nesses dois estados. Então, se mostrarmos que **não existe nenhum caminho com mais de $4n$ transições** a partir de um estado inicial qualquer (com a e b livres), toda a execução tem de acabar em `stop` ou `error` ao fim de, no máximo, $4n$ passos.

Pede-se ao Z3 caminhos de $4n$ e de $4n + 1$ transições:

- $4n$ transições: `sat`, ou seja, existe um caminho tão longo (o pior caso é y com todos os bits a 1);
- $4n + 1$ transições: `unsat`, ou seja, já não existe nenhum.

O resultado é para $n = 8$, o valor definido no início.

Testes Unitários

Para executar os testes automatizados em `unittest` criados no ficheiro de testes. Por causa da organização e ter a certeza que os testes correm e a implementação está certa, os testes são corridos apartir de um ficheiro à parte do notebook marimo com apenas a lógica(TP3.py).

(`test_mult.py`):

```
--powershell

python -m unittest test_mult.py

--ubuntu e mac


python3 -m unittest test_mult.py
```

Uso de LLM

link: https://chatgpt.com/share/6aea35f2-da84-83ed-9e06-8666f05f5166